

POWER BI

Interview Guide



Rajat Saxena
(SENIOR ANALYST)

Microsoft Power BI

Interview Question Bank

-By Rajat Saxena

Q1: What is Power BI and what are its main components?

Answer: Power BI is a business analytics service by Microsoft that provides interactive visualizations and business intelligence capabilities. Its main components are:

1. **Power BI Desktop** - Free desktop application for creating reports
2. **Power BI Service** - Cloud-based service for sharing and collaboration
3. **Power BI Mobile** - Mobile apps for iOS, Android, and Windows
4. **Power BI Report Builder** - For creating paginated reports
5. **Power BI Embedded** - For embedding reports in applications

Explanation: Understanding these components helps you choose the right tool for different scenarios. For example, use Desktop for development, Service for sharing, and Mobile for on-the-go access.

Q2: What is the difference between Power BI Desktop and Power BI Service?

Answer:

Aspect	Power BI Desktop	Power BI Service
Purpose	Report development	Report sharing & collaboration
Cost	Free	Freemium model (free with limitations)

Aspect	Power BI Desktop	Power BI Service
Data Refresh	Manual/limited	Scheduled refresh available
Sharing	Local files only	Web-based sharing
Data Modeling	Full capabilities	Limited modeling

Explanation: Desktop is your development environment where you build data models and reports. Service is where you publish, share, and schedule refreshes. Interviewers want to ensure you know when to use each tool.

Q3: What are the different types of connections in Power BI?

Answer:

1. **Import Mode** - Data is imported into Power BI (most common)
2. **DirectQuery** - Live connection to source database
3. **Live Connection** - Connect to published datasets or Analysis Services
4. **Dual Mode** - Combination of Import and DirectQuery

Explanation: Import mode is best for performance but has data size limits. DirectQuery is for real-time data but has performance limitations. Choose based on data size, freshness requirements, and source system capabilities.

Q4: What is DAX and when would you use it?

Answer: DAX (Data Analysis Expressions) is a formula language used in Power BI, Excel Power Pivot, and Analysis Services for calculations and data analysis.

Common uses:

- Creating calculated columns
- Defining measures
- Creating calculated tables
- Time intelligence functions

Example:

```
1 Total Sales = SUM(Sales[Amount])
2
```

Explanation: DAX is essential for creating business logic that isn't in your source data. It's similar to Excel formulas but optimized for relational data models.

Q5: What is the difference between a measure and a calculated column?

Answer:

Aspect	Measure	Calculated Column
Storage	Calculated at query time	Stored in the model
Performance	Better for large datasets	Increases model size
Use Case	Aggregations, KPIs	Row-level calculations
Context	Respects filter context	Fixed at row level

Example:

```
1 -- Measure (dynamic calculation)
2 Total Profit = SUM(Sales[Profit])
3
4 -- Calculated Column (static per row)
5 Profit Margin = Sales[Profit] / Sales[Revenue]
6
```

Explanation: Measures are dynamic and respond to filters/slicers. Calculated columns are static and stored, which increases file size. Use measures for aggregations, calculated columns for row-level logic needed in filters.

Q6: What is Power Query and what is it used for?

Answer: Power Query is the data transformation and preparation engine in Power BI (also in Excel). It's used for:

- Connecting to various data sources
- Cleaning and transforming data
- Merging and appending queries
- Pivoting/unpivoting data
- Handling errors and missing values

Explanation: Power Query uses M language and provides a user-friendly interface for ETL (Extract, Transform, Load) processes. It's crucial for data preparation before analysis.

Q7: What are the different types of visualizations available in Power BI?

Answer: Common visualization types include:

1. **Basic Charts** - Bar, column, line, area, pie
2. **Advanced Charts** - Scatter, bubble, waterfall, funnel
3. **Maps** - Filled map, shape map, ArcGIS maps
4. **Tables & Matrix** - For detailed data
5. **Cards** - Single number/KPI displays
6. **Slicers** - For filtering
7. **Custom Visuals** - From marketplace

Explanation: Choosing the right visualization depends on your data story. Bar charts compare categories, line charts show trends, scatter plots show relationships, etc.

Q8: What is a slicer and how does it work?

Answer: A slicer is a visual filter that allows users to filter report data interactively.

Types of slicers:

1. **List slicer** - Dropdown or list format
2. **Date slicer** - For date filtering
3. **Numeric range slicer** - For number ranges
4. **Relative date slicer** - For time-based filtering

Explanation: Slicers apply filters to all visuals on a page (by default) or selected visuals. They provide intuitive user interaction and are essential for dashboard usability.

Q9: What is the difference between "Filter on this visual" and "Filter on all pages"?

Answer:

- **Filter on this visual** - Applies only to that specific visual
- **Filter on all pages** - Applies to all visuals in the report
- **Filter on this page** - Applies to all visuals on current page

Explanation: This granular control allows you to create targeted filtering. For example, you might have a global date filter (all pages) and a product category filter specific to one visual.

Q10: How do you handle missing or null values in Power BI?

Answer: Several approaches:

1. **Power Query** - Use "Replace Values" or "Fill Down"
2. **DAX** - Use functions like IF(ISBLANK([Value]), 0, [Value])
3. **Visual level** - Use "Show items with no data" option
4. **Modeling** - Create a "Unknown" category in dimension tables

Example in Power Query:

```
1 = Table.ReplaceValue(Source, null, 0, Replacer.ReplaceValue, {"Column1"})
2
```

Explanation: Handling nulls prevents calculation errors and ensures accurate aggregations. The approach depends on whether nulls represent missing data or zero values.

Q11: What is a hierarchy and how do you create one?

Answer: A hierarchy organizes data into levels (e.g., Year → Quarter → Month → Day).

To create:

1. Right-click a field in Fields pane
2. Select "New hierarchy"
3. Drag other fields into the hierarchy

Explanation: Hierarchies enable drill-down analysis. Users can start at a high level (Year) and drill to details (Day). Essential for time analysis and organizational structures.

Q12: What is the difference between "And" and "Or" in filter logic?

Answer:

- **And** - All conditions must be true
- **Or** - Any condition can be true

Example:

- Region = "East" AND Sales > 1000 - Both must be true
- Region = "East" OR Region = "West" - Either can be true

Explanation: Understanding Boolean logic is crucial for complex filtering. Power BI uses this in visual filters, DAX expressions, and query filtering.

Q13: What is a bookmark and how is it used?

Answer: Bookmarks capture the current state of a report page (filters, slicers, visual states).

Common uses:

1. Creating interactive stories
2. Saving specific views
3. Building navigation systems
4. Showing/hiding visuals dynamically

Explanation: Bookmarks enable guided analytics. You can create a "story" that walks users through different insights by sequencing bookmarks.

Q14: What is the "Analyze in Excel" feature?

Answer: This feature allows Excel users to connect to Power BI datasets and create PivotTables charts, and analysis using Excel's familiar interface.

How to use:

1. In Power BI Service, select dataset
2. Click "Analyze in Excel"
3. Open the .odc file in Excel

Explanation: This bridges Power BI and Excel, leveraging Excel's advanced analysis capabilities while using Power BI's data model. Great for users comfortable with Excel.

Q15: How do you share a Power BI report?

Answer: Sharing methods:

1. **Publish to Power BI Service** - From Desktop
2. **Share dashboard/report** - Via Service interface
3. **Publish to web** - Public sharing (careful with sensitive data)
4. **Apps** - Package reports for groups
5. **Embed** - In SharePoint, Teams, or websites

Explanation: Different sharing methods serve different security and collaboration needs. Apps are best for organizational distribution, while embed is for integration.

Q16: What is a data gateway and when do you need it?

Answer: A data gateway is a bridge that enables secure data transfer between on-premises data sources and Power BI Service.

You need it when:

- Data sources are on-premises
- Using DirectQuery or Live Connection
- Scheduling data refreshes from on-prem sources

Types:

1. **Personal mode** - For one user
2. **Enterprise mode** - For multiple users/data sources

Explanation: Gateways are crucial for hybrid environments. They maintain security by keeping data behind firewall while enabling cloud access.

Q17: What is the difference between "Enter Data" and "Get Data"?

Answer:

- **Get Data** - Connect to external data sources
- **Enter Data** - Manually create a table by typing/pasting data

Explanation: "Enter Data" is useful for small reference tables, testing, or when you don't have access to the source system. "Get Data" is for connecting to actual data sources.

Q18: How do you format numbers in Power BI?

Answer: Formatting options:

1. **Visual level** - Format pane for each visual
2. **Data model level** - Column properties
3. **DAX** - Using FORMAT() function

Common formats:

- Currency, Percentage, Decimal places
- Custom formats (e.g., "0.0%", "\$#,##0")

Explanation: Consistent formatting improves readability. Set formats at the model level for consistency across all visuals using that column.

Q19: What is a tooltip in Power BI?

Answer: Tooltips are informational pop-ups that appear when hovering over data points. You can create enhanced tooltips as report pages.

To create enhanced tooltip:

1. Create a new report page
2. Set page size as "Tooltip"
3. Add relevant visuals
4. Set as tooltip for main visuals

Explanation: Enhanced tooltips provide additional context without cluttering the main view. They're great for showing details on demand.

Q20: How do you rename columns or tables?

Answer: Multiple ways:

1. **Power Query** - Right-click column → Rename
2. **Model view** - Double-click column/table name
3. **DAX** - Using SELECTCOLUMNS() for calculated tables

Best Practice: Rename in Power Query for consistency. Avoid spaces/special characters for easier DAX referencing.

Q21: What is filter context and row context in DAX?

Answer:

- **Filter Context** - Filters applied by visuals, slicers, or DAX functions
- **Row Context** - Context for each row in a table (in calculated columns or ITERATE functions)

Example:

```
1  -- Filter context affects this measure
2  Total Sales = SUM(Sales[Amount])  -- Respects current filters
3
4  -- Row context in calculated column
5  Row Sales = Sales[Quantity] * Sales[Price]  -- Calculated per row
6
```

Explanation: Understanding context is the most important DAX concept. Filter context determines which rows are considered in calculations. Row context is like an implicit loop through table rows.

Q22: What are CALCULATE and FILTER functions in DAX?

Answer:

- **CALCULATE** - Modifies filter context for a calculation
- **FILTER** - Returns a table filtered by a condition

Examples:

```
1  -- CALCULATE changes context
2  West Sales = CALCULATE(SUM(Sales[Amount]), Sales[Region] = "West")
3
4  -- FILTER creates filtered table
5  High Value Sales = CALCULATE(SUM(Sales[Amount]),
6      FILTER(Sales, Sales[Amount] > 1000))
7
```

Explanation: CALCULATE is the most important DAX function. It allows you to override existing filters. FILTER is often used inside CALCULATE for complex conditions.

Q23: What are time intelligence functions? Provide examples.

Answer: Time intelligence functions perform calculations over time periods.

Common functions:

```

1  -- Previous period
2  Sales PY = CALCULATE(SUM(Sales[Amount]), SAMEPERIODLASTYEAR(Date[Date]))
3
4  -- Year-to-date
5  Sales YTD = TOTALYTD(SUM(Sales[Amount]), Date[Date])
6
7  -- Moving average (30 days)
8  Sales MA30 =
9  AVERAGEX(
10     DATESINPERIOD(Date[Date], LASTDATE(Date[Date]), -30, DAY),
11     [Total Sales]
12  )
13

```

Explanation: These functions require a properly structured date table. They're essential for trend analysis, comparisons, and period calculations.

Q24: How do you create a date table in Power BI?

Answer: Several methods:

1. Using DAX:

```

1  DateTable =
2  ADDCOLUMNS(
3      CALENDAR(DATE(2020,1,1), DATE(2025,12,31)),
4      "Year", YEAR([Date]),
5      "Quarter", "Q" & QUARTER([Date]),
6      "Month", FORMAT([Date], "MMMM"),
7      "MonthNumber", MONTH([Date]),
8      "DayOfWeek", FORMAT([Date], "dddd")
9  )
10

```

2. Using Power Query (M):

```
let
    StartDate = #date(2020, 1, 1),
    EndDate = #date(2025, 12, 31),
    NumberOfDays = Duration.Days(EndDate - StartDate),
    DateList = List.Dates(StartDate, NumberOfDays, #duration(1,0,0,0)),
    #"Converted to Table" = Table.FromList(DateList, Splitter.SplitByNothing(), {"Date"}),
    #"Added Columns" = Table.AddColumn(#"Converted to Table", "Year", each Date.Year([Date]))
in
    #"Added Columns"
```

Explanation: A proper date table is crucial for time intelligence. It should have continuous dates, no gaps, and include all necessary date attributes.

Q25: What is the difference between SUM and SUMX?

Answer:

- **SUM** - Aggregates values in a column
- **SUMX** - Iterates over a table and evaluates an expression for each row

Examples:

```

1  -- SUM (simple aggregation)
2  Total Revenue = SUM(Sales[Revenue])
3
4  -- SUMX (row-by-row calculation)
5  Total Profit = SUMX(Sales, Sales[Revenue] - Sales[Cost])
6
7  -- SUMX with related table
8  Customer Sales = SUMX(Customers,
9      CALCULATE(SUM(Sales[Amount])))
10

```

Explanation: Use SUM for simple column totals. Use SUMX when you need to calculate something per row before summing, or when working with related tables.

Q26: What are variables in DAX and why use them?

Answer: Variables store intermediate results in DAX expressions.

Syntax:

```

1  Measure With Variables =
2  VAR TotalSales = SUM(Sales[Amount])
3  VAR TotalCost = SUM(Sales[Cost])
4  RETURN
5  TotalSales - TotalCost
6

```

Benefits:

1. Improve performance (calculate once, reuse)
2. Enhance readability
3. Simplify debugging
4. Organize complex calculations

Explanation: Variables make DAX more maintainable. They're evaluated once and can be referenced multiple times in the RETURN statement.

Q27: How do you handle many-to-many relationships?

Answer: Approaches for many-to-many:

1. Bridge Table:

```
1  -- Create a bridge table
2  BridgeTable =
3  SUMMARIZE(
4      Sales,
5      Sales[ProductID],
6      Sales[CustomerID]
7  )
8
```

2. DAX Patterns:

```
1  Sales by Customer =
2  CALCULATE(
3      SUM(Sales[Amount]),
4      CROSSFILTER(Sales[CustomerID], Customer[CustomerID], Both)
5  )
6
```

3. Using TREATAS:

```

1 Virtual Relationship =
2 CALCULATE(
3     SUM(Sales[Amount]),
4     TREATAS(VALUES(Table1[ID]), Table2[ID])
5 )
6

```

Explanation: Many-to-many requires careful modeling. Bridge tables are the cleanest solution. DAX functions like CROSSFILTER and TREATAS can handle specific scenarios.

Q28: What is the difference between ALL and ALLEXCEPT?

Answer:

- **ALL** - Removes all filters from a table or columns
- **ALLEXCEPT** - Removes all filters except specified columns

Examples:

```

1 -- ALL: Total across everything
2 Total All Products = CALCULATE(SUM(Sales[Amount]), ALL(Products))
3
4 -- ALLEXCEPT: Total by category (keep category filter)
5 Total by Category =
6 CALCULATE(
7     SUM(Sales[Amount]),
8     ALLEXCEPT(Sales, Products[Category])
9 )
10

```

Explanation: These functions modify filter context. ALL is often used in % of total calculations. ALLEXCEPT is useful for keeping certain grouping levels.

Q29: What is the difference between RELATED and RELATEDTABLE?

Answer:

- **RELATED** - Fetches a single related value from another table (many-to-one)
- **RELATEDTABLE** - Returns a table of related rows (one-to-many)

Examples:

```
1  -- RELATED: Get category name for each product
2  Product Category = RELATED(ProductCategory[CategoryName])
3
4  -- RELATEDTABLE: Count orders for each customer
5  Order Count = COUNTROWS(RELATEDTABLE(Orders))
6
```

Explanation: Use RELATED in calculated columns to bring in attributes from dimension tables. Use RELATEDTABLE when you need to work with multiple related rows.

Q30: How do you create dynamic titles in Power BI?

Answer: Use DAX measures in title text boxes:

Method:

1. Create a measure for the title:

```
1  Dynamic Title =
2  "Sales Report - " &
3  SELECTEDVALUE('Date'[Year], "All Years") &
4  " | Region: " &
5  SELECTEDVALUE(Geography[Region], "All Regions")
6
```

2. Add a text box to report
3. Insert the measure: Click text box → Type = → Select measure

Explanation: Dynamic titles update based on filters, making reports more contextual. Useful for dashboards that users filter frequently.

Q31: What is the difference between CONCATENATE and CONCATENATEX?

Answer:

- **CONCATENATE** - Joins two text strings
- **CONCATENATEX** - Iterates over a table and concatenates values

Examples:

```
1  -- CONCATENATE
2  Full Name = CONCATENATE(Customer[FirstName], " ", Customer[LastName])
3
4  -- CONCATENATEX (list top 3 products)
5  Top Products =
6  CONCATENATEX(
7      TOPN(3, VALUES(Products[ProductName]), [Total Sales]),
8      Products[ProductName],
9      ", "
10 )
11
```

Explanation: CONCATENATEX is powerful for creating comma-separated lists or combining values from multiple rows. Great for tooltips or labels.

Q32: How do you handle currency conversion in Power BI?

Answer: Approaches for multi-currency:

1. Using Exchange Rate Table:

```
1 Sales in USD =
2 SUMX(
3     Sales,
4     Sales[Amount] *
5     LOOKUPVALUE(
6         ExchangeRates[Rate],
7         ExchangeRates[FromCurrency], Sales[Currency],
8         ExchangeRates[ToCurrency], "USD",
9         ExchangeRates[Date], Sales[Date]
10    )
11 )
12
```

2. Using DAX Measures with Selection:

```
1 Converted Amount =
2 VAR SelectedCurrency = SELECTEDVALUE(CurrencyTable[Currency], "USD")
3 RETURN
4 SUMX(
5     Sales,
6     Sales[Amount] *
7     CALCULATE(
8         VALUES(ExchangeRates[Rate]),
9         ExchangeRates[ToCurrency] = SelectedCurrency
10    )
11 )
12
```

Explanation: Currency conversion requires an exchange rate table with date, from/to currencies, and rate. Use LOOKUPVALUE or relationships with appropriate filtering.

Q33: What is the difference between VALUES and DISTINCT?

Answer:

- **VALUES** - Returns unique values including blanks (if present)
- **DISTINCT** - Returns unique values excluding blanks

Examples:

```
1  -- VALUES (includes blank if exists)
2  Unique Customers = COUNTROWS(VALUES(Customers[CustomerID]))
3
4  -- DISTINCT (excludes blank)
5  Non-blank Products = COUNTROWS(DISTINCT(Products[ProductName]))
6
```

Explanation: Use VALUES when you need to preserve the data model's behavior (including blanks). Use DISTINCT when blanks should be ignored.

Q34: How do you create a what-if parameter?

Answer: What-if parameters allow users to adjust variables dynamically.

Steps:

1. Modeling tab → New parameter
2. Configure range, increment, data type
3. Power BI creates a table and slicer automatically

Example Usage (Discount Scenario):

```
1 Adjusted Sales =  
2 [Total Sales] * (1 - 'Discount Parameter'[Discount Value])  
3
```

Explanation: What-if parameters enable scenario analysis. Users can adjust parameters via slicers to see immediate impact on calculations.

Q35: What is the difference between UNION and INTERSECT in DAX?

Answer:

- **UNION** - Combines tables (removes duplicates)
- **INTERSECT** - Returns common rows between tables

Examples:

```
1 -- UNION: Combine two customer lists  
2 All Customers =  
3 UNION(  
4     SELECTCOLUMNS(Customers2019, "Customer", Customers2019[Name]),  
5     SELECTCOLUMNS(Customers2020, "Customer", Customers2020[Name])  
6 )  
7  
8 -- INTERSECT: Find returning customers  
9 Returning Customers =  
10 INTERSECT(  
11     VALUES(Customers2019[CustomerID]),  
12     VALUES(Customers2020[CustomerID])  
13 )  
14
```

Explanation: UNION is for combining data. INTERSECT is for finding overlaps. Both require tables with the same structure.

Q36: How do you optimize Power BI performance?

Answer: Performance optimization strategies:

Data Model:

1. Reduce columns to only necessary ones
2. Use integers instead of strings for keys
3. Avoid bi-directional relationships
4. Use star schema design

DAX:

1. Use variables to avoid repeated calculations
2. Avoid calculated columns when possible
3. Use CALCULATE efficiently
4. Minimize iterator functions (X functions) on large tables

Visuals:

1. Limit visuals on a page
2. Use aggregation instead of detail
3. Disable visual interactions when not needed
4. Use summarized tables for large datasets

Power Query:

1. Filter early in query steps
2. Remove unnecessary columns
3. Use query folding when possible
4. Avoid merging large tables if possible

Explanation: Performance is multi-layered. Start with data model optimization, then DAX, then visuals. Use Performance Analyzer (View tab) to identify bottlenecks.

Q37: What is query folding and why is it important?

Answer: Query folding is when Power Query pushes transformation steps back to the source database instead of processing locally.

Benefits:

1. Better performance (database does the work)
2. Reduced data transfer
3. Leverages database optimizations

To check if folding occurs:

1. Right-click step → View Native Query
2. If disabled, folding isn't happening

Operations that prevent folding:

- Adding custom columns with M functions
- Certain data type changes
- Some merging operations

Explanation: Always aim for query folding when working with databases. It significantly improves refresh performance, especially with large datasets.

Q38: How do you create a cumulative total in Power BI?

Answer: Several methods:

1. Using DAX (Running Total):

```
1 Cumulative Sales =  
2 CALCULATE(  
3     SUM(Sales[Amount]),  
4     FILTER(  
5         ALLSELECTED('Date'),  
6         'Date'[Date] <= MAX('Date'[Date])  
7     )  
8 )  
9
```

2. Using Quick Measure:

- Select line chart
- Quick measures → Running total

3. Using Window Functions in Power Query (if source supports):

```
1 = Table.Buffer(Source)
2 // Then add index and custom column
3
```

Explanation: Cumulative totals show progression over time. The DAX method is most common and works with any date hierarchy.

Q39: What is the difference between EARLIER and EARLIEST?

Answer:

- **EARLIER** - Refers to the previous row context
- **EARLIEST** - Refers to the outermost row context

Example (Rank within category):

```
1 Rank in Category =
2 COUNTROWS(
3     FILTER(
4         Products,
5         Products[Category] = EARLIER(Products[Category]) &&
6         Products[Sales] > EARLIER(Products[Sales])
7     )
8 ) + 1
9
```

Explanation: These functions are used in nested row contexts (calculated columns with filters). EARLIER is more common; EARLIEST is for deeply nested contexts.

Q40: How do you handle slowly changing dimensions (SCD)?

Answer: Approaches for SCD Type 2:

1. Using Power Query:

```
1 // Merge current and historical data
2 // Add effective date columns
3 // Handle active flag
4
```

2. Using DAX:

```
1 Current Employee =
2 CALCULATE(
3     VALUES(Employee[Name]),
4     Employee[EndDate] = DATE(9999,12,31) // Active record
5 )
6
7 Historical Sales =
8 CALCULATE(
9     SUM(Sales[Amount]),
10    FILTER(
11        Employee,
12        Sales[Date] >= Employee[StartDate] &&
13        Sales[Date] <= Employee[EndDate]
14    )
15 )
16
```

3. Source Level: Handle in database view

Explanation: SCD Type 2 maintains history by adding new rows with date ranges. The key is linking facts to the correct dimension version based on dates.

Q41: What is the difference between GROUPBY and SUMMARIZE?

Answer:

- **SUMMARIZE** - Groups by columns and can add calculated columns

- **GROUPBY** - Groups by columns and can perform aggregations using `CURRENTGROUP()`

Examples:

```
1  -- SUMMARIZE
2  Sales Summary =
3  SUMMARIZE(
4      Sales,
5      Products[Category],
6      "Total Sales", SUM(Sales[Amount]),
7      "Average Price", AVERAGE(Sales[Price])
8  )
9
10 -- GROUPBY (more efficient for complex calculations)
11 Sales Grouped =
12 GROUPBY(
13     Sales,
14     Products[Category],
15     "Total Sales", SUMX(CURRENTGROUP(), Sales[Amount])
16 )
17
```

Explanation: GROUPBY is generally more efficient and should be preferred for grouping operations. SUMMARIZE has some limitations with complex calculations.

Q42: How do you create a dynamic measure selection?

Answer: Allow users to choose which measure to display:

Steps:

1. Create a measure table:

```
1 MeasuresTable =  
2 DATATABLE(  
3     "MeasureName", STRING,  
4     "MeasureKey", INTEGER,  
5     {  
6         {"Sales", 1},  
7         {"Profit", 2},  
8         {"Quantity", 3}  
9     }  
10 )  
11
```

2. Create selected measure:

```
1 Selected Measure =  
2 SWITCH(  
3     SELECTEDVALUE(MeasuresTable[MeasureKey]),  
4     1, [Total Sales],  
5     2, [Total Profit],  
6     3, [Total Quantity],  
7     BLANK()  
8 )  
9
```

3. Use slicer on MeasureName, display Selected Measure

Explanation: This pattern creates flexible reports where users can switch between metrics without changing visuals.

Q43: What is the difference between TREATAS and INTERSECT?

Answer:

- **TREATAS** - Creates a virtual relationship between tables
- **INTERSECT** - Finds common rows between tables

Example of TREATAS:

```
1 Filtered Sales =  
2 CALCULATE(  
3     SUM(Sales[Amount]),  
4     TREATAS(VALUES(SelectedProducts[ProductID]), Products[ProductID])  
5 )  
6
```

Explanation: TREATAS is for filtering, INTERSECT is for set operations. TREATAS is often used in many-to-many scenarios or disconnected tables.

Q44: How do you create a dynamic axis in charts?

Answer: Allow users to choose which column to use as X-axis:

Method:

1. Create parameter table for axis options
2. Create measure for dynamic axis:

```
1 Dynamic Axis =  
2 SWITCH(  
3     SELECTEDVALUE(AxisSelector[AxisChoice]),  
4     "Category", SELECTEDVALUE(Products[Category]),  
5     "Region", SELECTEDVALUE(Geography[Region]),  
6     "Month", SELECTEDVALUE('Date'[MonthName])  
7 )  
8
```

3. Use this measure as chart axis

Explanation: Dynamic axes make reports more flexible. Users can change perspective without creating multiple visuals.

Q45: What is the difference between ISFILTERED and ISCROSSFILTERED?

Answer:

- **ISFILTERED** - Returns TRUE if column has direct filter
- **ISCROSSFILTERED** - Returns TRUE if column has direct or cross filter

Examples:

```
1 -- Check if Region is filtered  
2 Is Region Filtered = ISFILTERED(Geography[Region])  
3  
4 -- Check if any filter affects Product (including via relationships)  
5 Is Product Filtered = ISCROSSFILTERED(Products[ProductID])  
6
```

Explanation: Use ISFILTERED to check for direct user filters. Use ISCROSSFILTERED to check for any filtering, including through relationships.

Q46: How do you create a Top N with Others category?

Answer: Show top N items and group the rest as "Others":

DAX Pattern:

```
Product Group =  
VAR TopN = 5  
VAR CurrentProduct = SELECTEDVALUE(Products[ProductName])  
VAR ProductRank = RANKX(ALL(Products), [Total Sales])  
RETURN  
IF(  
    ProductRank <= TopN,  
    CurrentProduct,  
    "Other Products"  
)  
  
Sales by Group =  
SUMX(  
    VALUES(Products[ProductName]),  
    IF(  
        [Product Group] = "Other Products",  
        [Total Sales],  
        IF([Product Group] = SELECTEDVALUE(Products[ProductName]), [Total Sales])  
    )  
)
```

Explanation: This pattern improves chart readability by focusing on important items while still showing the total picture.

Q47: What is the difference between USERELATIONSHIP and CROSSFILTER?

Answer:

- USERELATIONSHIP - Activates an inactive relationship for a calculation
- CROSSFILTER - Changes filter direction of a relationship

Examples:

```
1  -- USERELATIONSHIP (use delivery date instead of order date)
2  Delivery Sales =
3  CALCULATE(
4      SUM(Sales[Amount]),
5      USERELATIONSHIP(Sales[DeliveryDate], 'Date'[Date])
6  )
7
8  -- CROSSFILTER (enable filtering both ways)
9  BiDirectional Sales =
10 CALCULATE(
11     SUM(Sales[Amount]),
12     CROSSFILTER(Sales[CustomerID], Customer[CustomerID], Both)
13 )
14
```

Explanation: USERELATIONSHIP is for choosing between multiple date relationships. CROSSFILTER is for changing filter direction temporarily.

Q48: How do you handle time zones in Power BI?

Answer: Approaches for time zone conversion:

1. In Power Query:

```
1 = Table.TransformColumns(Source,  
2     {"UTC_Time", each DateTimeZone.ToLocal(_, type datetimetypezone)}}  
3
```

2. Using DAX:

```
1 Local Time =  
2 Sales[UTC_Time] + TIME(5, 30, 0)  -- For IST (UTC+5:30)  
3
```

3. Using Time Zone Table:

```
1 Local Time =  
2 Sales[UTC_Time] +  
3 LOOKUPVALUE(  
4     TimeZones[Offset],  
5     TimeZones[Timezone], SELECTEDVALUE(Location[Timezone])  
6 )  
7
```

Explanation: Store times in UTC, convert to local time in reports. Use a time zone table if dealing with multiple locations.

Q49: What is the difference between SELECTEDVALUE and HASONEVALUE?

Answer:

- SELECTEDVALUE - Returns the value if only one is selected, else returns alternate
- HASONEVALUE - Returns TRUE if only one value is selected

Examples:

```
1  -- SELECTEDVALUE (with default)
2  Selected Region = SELECTEDVALUE(Geography[Region], "All Regions")
3
4  -- HASONEVALUE (for validation)
5  Is Single Region =
6  IF(
7      HASONEVALUE(Geography[Region]),
8      "Single Region Selected",
9      "Multiple or No Regions"
10 )
11
```

Explanation: Use SELECTEDVALUE to get the filtered value. Use HASONEVALUE to check if a single value is selected before using it.

Q50: How do you create a dynamic ranking measure?

Answer: Create ranks that respond to filters:

```
1 Sales Rank =  
2 IF(  
3     HASONEVALUE(Products[ProductName]),  
4     RANKX(  
5         ALLSELECTED(Products),  
6         [Total Sales],  
7         ,  
8         DESC,  
9         Dense  
10    )  
11 )  
12
```

Options for RANKX:

- **Skip** - Gaps in rank numbers (1, 3, 3, 4)
- **Dense** - No gaps (1, 2, 2, 3)
- **Desc/Asc** - Sort order

Explanation: Dynamic ranking updates as users filter. Use ALLSELECTED to rank within current filter context.

Q51: What is Tabular Editor and why use it?

Answer: Tabular Editor is a third-party tool for advanced Power BI modeling.

Key features:

1. **Bulk operations** - Edit multiple measures/tables
2. **DAX debugging** - Better than Power BI Desktop
3. **Version control** - Works with Git
4. **Best practice analyzer** - Checks model quality
5. **Advanced scripting** - C# scripts for automation

Common uses:

- Managing large models
- Applying consistent formatting
- Creating calculation groups
- Performance optimization

Explanation: While Power BI Desktop is good for development, Tabular Editor excels at model management and advanced features.

Q52: What are calculation groups and when to use them?

Answer: Calculation groups reduce measure duplication by applying time intelligence or format transformations.

Example scenario: Instead of creating:

- Sales YTD
- Sales PY
- Sales YoY%

Create one calculation group "Time Intelligence" with:

- YTD
- PY
- YoY%

Benefits:

1. Reduce measure count
2. Consistent calculations
3. Easier maintenance
4. Better performance

Implementation: Requires Tabular Editor or external tools (not native in Power BI Desktop yet).

Explanation: Calculation groups are essential for large models with many time intelligence measures. They follow DRY (Don't Repeat Yourself) principle.

Thank You for Reading

Thank you for taking the time to go through *Power BI Interview Guide*.
I truly hope this guide helps you strengthen your Power BI concepts, build confidence, and perform better in interviews.

This guide was created with one clear goal — **to genuinely help job seekers prepare smarter and land the right opportunities**. If even a small part of this book adds value to your preparation, it means a lot to me.

Your learning journey doesn't have to stop here. I regularly share **interview questions, job updates, preparation tips, and career guidance** on my platforms. Feel free to connect and stay in touch:

 [LinkedIn](#)

 [WhatsApp Channel](#)

 [Telegram Channel](#)

If you found this guide helpful, do share it with fellow job seekers who might benefit from it.

Wishing you **all the very best for your Power BI interviews and career growth**



Warm regards,

[Rajat Saxena](#)